

# Grille d'autoévaluation - Semaine 3

420-W30-SF · Branches Git, dépendances et injection de dépendances · Automne 2026

Nom (facultatif)

Date de révision

## Comment utiliser cette grille

Cachez vos notes, puis cochez seulement ce que vous pouvez expliquer ou réaliser sans suivre une recette. Commencez votre révision par une case non cochée.

### 1. Reconnaître les dépendances

Je suis capable de :

- ☐ repérer qu'une classe construit elle-même un collaborateur concret;
- ☐ expliquer pourquoi cette création produit un couplage fort;
- ☐ distinguer la logique d'un objet de la création de ses dépendances;
- ☐ identifier la dépendance à remplacer pour rendre un test simple et déterministe;
- ☐ expliquer pourquoi une interface étroite facilite la substitution;
- ☐ choisir un nom d'interface qui décrit une capacité utile au client.

Dépendance concrète que je pourrais rendre substituable

### 2. Injection manuelle par constructeur

Je suis capable de :

- ☐ recevoir une dépendance par un paramètre de constructeur;
- ☐ conserver cette dépendance dans une variable d'objet privée préfixée m\_;
- ☐ refuser une dépendance null au moment de la construction;
- ☐ assembler manuellement une implantation et son client dans Program.cs;
- ☐ expliquer pourquoi le constructeur rend la dépendance visible;
- ☐ distinguer l'injection de dépendances de l'utilisation d'un conteneur.

Constructeur injecté que je peux écrire sans aide

### 3. Tester un objet qui reçoit une dépendance

Je suis capable de :

- ☐ fournir une implantation mémoire ou une doublure simple au système testé;
- ☐ construire directement le système testé sans utiliser le conteneur;
- ☐ séparer le test en Arranger, Agir et Auditer;
- ☐ mémoriser un appel et ses arguments dans une doublure;
- ☐ auditer l'interaction après l'action;
- ☐ vérifier à la fois un changement d'état et une interaction lorsque le cas l'exige;
- ☐ expliquer pourquoi une vraie sortie console ne convient pas à ce test unitaire.

Interaction que mon test doit pouvoir observer

### 4. Inversion de contrôle et conteneur .NET

Je suis capable de :

- ☐ expliquer qui contrôle la création des objets avec et sans conteneur;
- ☐ distinguer inversion de contrôle, inversion des dépendances et injection de dépendances;
- ☐ enregistrer une interface et son implantation avec AddScoped;
- ☐ enregistrer le service qui reçoit cette interface;
- ☐ appeler Build, CreateScope et GetRequiredService au point de composition;
- ☐ garder le conteneur dans Program.cs;
- ☐ expliquer pourquoi Application et le domaine ne doivent pas recevoir IServiceProvider.

Rôle du point de composition, dans mes propres mots

### 5. Cycles de vie

Je suis capable de :

- ☐ distinguer les cycles de vie Transient, Scoped et Singleton;
- ☐ expliquer ce qu'une portée représente dans l'application de la semaine;
- ☐ prédire si deux résolutions dans une même portée donnent la même instance;
- ☐ expliquer pourquoi une instance Scoped ne doit pas être conservée par un Singleton;
- ☐ choisir un cycle de vie et justifier ce choix selon l'état de l'objet;
- ☐ reconnaître que le conteneur automatise l'assemblage, mais ne remplace pas la conception.

Cycle de vie que je dois revoir et raison

## 6. Branches locales et distantes

Je suis capable de :

- ☐ expliquer qu'une branche est un nom qui pointe vers un commit;
- ☐ vérifier main avant de créer une branche;
- ☐ créer et activer une branche avec git switch -c;
- ☐ publier une branche avec git push -u origin;
- ☐ récupérer les informations distantes avec git fetch;
- ☐ créer une branche locale qui suit une branche distante;
- ☐ passer d'une branche à l'autre sans perdre de modifications;
- ☐ lire git status et le graphe de l'historique avant une fusion.

Suite de commandes Git que je dois encore pratiquer

## 7. Fusionner et résoudre un conflit

Je suis capable de :

- ☐ identifier la branche de destination avant d'exécuter git merge;
- ☐ distinguer une avance rapide d'un commit de fusion;
- ☐ expliquer pourquoi une fusion peut échouer même si les deux branches sont valides;
- ☐ repérer les marqueurs d'un conflit dans un fichier;
- ☐ choisir le contenu final plutôt que conserver automatiquement une version complète;
- ☐ terminer une résolution avec git add et git commit;
- ☐ exécuter les tests après une fusion et avant de publier;
- ☐ intégrer une fonctionnalité de fonctionnalité/... vers dev, puis de dev vers main.

Conflit simple que je pourrais expliquer à une autre personne

## 8. Défis de vérification

Sans consulter immédiatement les notes, je peux :

- ☐ transformer une dépendance concrète en interface injectée;
- ☐ assembler les objets manuellement, puis avec le conteneur;
- ☐ tester le service directement avec une doublure simple;
- ☐ justifier un cycle de vie Scoped;
- ☐ réaliser le parcours fonctionnalité/... vers dev, puis vers main;
- ☐ résoudre un conflit et confirmer que la solution fonctionne encore.

## 9. Mon plan de révision

Notion prioritaire et prochaine action concrète

Question à apporter en classe